Machine Coding: Tooltip on Hover (and Focus)

- **Problem:** Build a reusable `<Tooltip content placement>` that wraps any trigger. It shows on mouse hover (after a short delay) **and** on keyboard focus, hides on mouse leave / blur / Escape, can be hovered itself, supports top / bottom / left / right, flips to the other side when there's no room, stays inside the screen, never gets clipped by `overflow: hidden` parents, and is announced by screen readers.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Triggers, placements, a11y rules
Use cases + API + state	5 min	`<Tooltip>` contract, `open` + `position`
CSS-positioned tooltip on hover (simplest)	8 min	Working tooltip
Focus/blur, Escape, delays, hoverable tooltip	10 min	Accessible behaviour
Portal + fixed positioning + pure `computePosition`	15 min	Never clipped, flips, clamps
Arrow, reposition on scroll/resize, animation	7 min	Polished
Test + wrap-up	10 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"A tooltip positioned with CSS inside its parent breaks as soon as a parent has `overflow: hidden` or the trigger is*

near the screen edge. So I render it in a portal, measure the trigger with `getBoundingClientRect`*, and compute the position in a pure function I can test: preferred side, flip if it doesn't fit, clamp inside the viewport."*

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- `<Tooltip content="…" placement="top|bottom|left|right" delay={300}>{trigger}</Tooltip>`.
- Show: hover (after `delay`), keyboard focus (immediately). Hide: mouse leave (after 100 ms grace), blur, Escape.
- The pointer can move from the trigger onto the tooltip without it disappearing.
- Flip to the opposite side if the preferred side doesn't fit; clamp inside the viewport; arrow keeps pointing at the trigger.
- Works inside `overflow: hidden` containers and follows scrolling/resizing.
- Long text wraps at a max width.

Non-functional requirements

Concern	What it means here
Performance	Hidden tooltips cost nothing (no listeners); scroll/resize listeners exist only while open; layout measured once per open/scroll.
Scalability	One self-contained component per trigger; pure `computePosition` reusable for popovers/menus.
Error handling	Timers cleared on unmount; refs checked before measuring; tiny viewports never produce negative positions.
Accessibility	`role="tooltip"`, `aria-describedby` on the trigger (always resolvable), shows on focus not only hover, Escape to dismiss, hoverable and persistent (WCAG 1.4.13), triggers are real buttons, reduced motion.
Reusability	Wraps any single focusable element via `cloneElement`.
Extensibility	Click-to-toggle mode for touch, rich content (→ popover), custom offset/colors.

Clarifying questions to ask the interviewer

1. Plain text only, or rich/interactive content? (Interactive content means a popover, not a tooltip.)
2. Which triggers: hover, focus, click/tap (mobile)? Any delay?
3. Which placements, and should it flip/auto-position near edges?
4. Can it be inside scrollable or `overflow: hidden` containers?
5. Should the user be able to hover the tooltip itself (to select or read long text)?
6. Can I use a positioning library (Floating UI), or should I write the positioning myself?

Assumptions we will build with

- Text-only content; hover delay 300 ms, focus immediate, hide grace 100 ms.
- Four placements with flip + viewport clamping; 8px gap and 8px screen padding.
- Rendered in a portal with `position: fixed`; repositioned on scroll (capture) and resize.
- The trigger is a focusable element (a button); the tooltip *describes* it (it doesn't replace its label).
- No library: positioning is a pure function with a self-check.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Mouse enters the trigger	Nothing for 300 ms, then the tooltip appears on the preferred side, centered
Mouse passes over quickly (< 300 ms)	Nothing shown (no flicker)
Mouse leaves	Hides after 100 ms
Mouse moves from the trigger onto the tooltip	Stays open (grace period cancelled)
Tab focuses the trigger	Shows immediately
Escape while open	Hides; focus stays on the trigger
Tab away (blur)	Hides
Trigger near the right edge, placement "right"	Flips to the left
Trigger near the top, placement "top"	Flips below
Trigger near a side edge with "top"	Tooltip shifted inside the screen; arrow still points at the trigger
Trigger inside an `overflow: hidden` box	Tooltip fully visible (portal)
Page scrolls while open	Tooltip follows the trigger
Very long text	Wraps at 240px max width
Screen reader focuses the trigger	"Security info, button, Your data is encrypted…"

3. Design the architecture

Module	Responsibility
`components/Tooltip.jsx`	Wraps the trigger; open state + timers; measures and positions; portal; ARIA wiring; Escape/scroll/resize listeners
`utils/position.js`	Pure `computePosition(triggerRect, tipSize, placement, viewport)` → `{ top, left, placement, arrow }`
`App`	Demo: 4 placements, icon button, long text, clipped box, edge case
`App.css`	Tooltip box, arrow per placement, hidden state, animation, reduced motion

```
<Tooltip content placement delay>
├── <span ref=triggerRef onMouseEnter/Leave onFocus/Blur>      ← listens for the trigger
│     └── cloneElement(children, { "aria-describedby": id })   ← e.g. <button>
└── createPortal(
      <div ref=tipRef id role="tooltip" hidden={!open} style={{top,left,--arrow}}
           onMouseEnter (cancel hide) onMouseLeave (hide)>{content}</div>,
      document.body)
```

```
useLayoutEffect(open) → measure both rects → computePosition() → setPosition
                      → listen scroll(capture)/resize/Escape while open
```

Start simple. First version: CSS-only tooltip (`position: absolute` + `bottom: 100%` inside a `position: relative` wrapper, shown on `:hover` / `:focus-within`). Then move to state + portal + measured positioning once the edge/clipping cases come up.

4. Design the state / data model

State	Where	Purpose
`open`	Tooltip	Is the tooltip shown
`position`	Tooltip	`{ top, left, placement, arrow }` from `computePosition`
`timer` (ref)	Tooltip	Pending show/hide timeout (one at a time)
`triggerRef` , `tipRef` (refs)	Tooltip	DOM nodes to measure

```
const [open, setOpen] = useState(false);
const [position, setPosition] = useState({ top: 0, left: 0, placement, arrow: 0 });
const timer = useRef(null);

const schedule = (nextOpen, wait) => {           // one timer for both show and hide
  clearTimeout(timer.current);
  timer.current = setTimeout(() => setOpen(nextOpen), wait);
};
```

Senior note

- **The tooltip owns its own open state:** the first version kept `showToolTip` in the parent `App` and wrapped the trigger in hover handlers by hand, so every tooltip needed boilerplate in its parent. Encapsulating it makes `<Tooltip>` drop-in.
- **One timer, not two:** a single `schedule(open, wait)` that always clears the previous timeout makes "enter, leave, enter" races impossible.
- **`position.placement` vs the `placement` prop:** the prop is the *preference*; the state holds the *actual* side after flipping (used for the arrow class).
- **Refs for DOM and timers:** measuring and timers don't need re-renders.

5. Design the data structures and contracts

Component API:

```
<Tooltip content="Save your changes" placement="top" delay={300}>
  <button type="button">Save</button>          {/* one focusable child */}
</Tooltip>
```

Positioning contract (pure, testable):

```
computePosition(
  trigger,        // DOMRect-like { top, left, right, bottom, width, height } in viewport px
  tip,            // { width, height }
  preferred,      // "top" | "bottom" | "left" | "right"
  viewport,       // { width, height }
  gap = 8, pad = 8
) → { top, left, placement /* actual */, arrow /* px along the facing side */ }
```

Example: trigger `{ left: 400, top: 400, width: 100, height: 40 }`, tip `120×30`, "top" → `{ top: 362, left: 390, placement: "top", arrow: 60 }`.

DOM output:

```
<button aria-describedby="r1">Save</button>
<!-- in <body> -->
<div id="r1" role="tooltip" class="tooltip tooltip-top" style="top:362px;left:390px;--arrow:60px">Save
your changes</div>
```

6. Trace the data flow

User hovers "Top", waits, then moves away:

```
mouseenter on the wrapper span → schedule(true, 300)
  ↓ 300 ms pass (no leave) → setOpen(true)
  ↓ render: tooltip loses `hidden` (still at the old position, not painted yet)
  ↓ useLayoutEffect (before paint): measure trigger + tip → computePosition(…, "top", viewport)
  ↓   fits above → top = trigger.top − h − 8, left centered, arrow at the center → setPosition
  ↓ re-render with the final position → browser paints once, in the right place (no flash)
  ↓ while open: scroll(capture)/resize → updatePosition; keydown Escape → setOpen(false)
mouseleave → schedule(false, 100) → (no re-enter within 100 ms) → setOpen(false) → hidden, listeners
removed
```

7. Important technical challenges

Challenge 1: clipping and stacking (portal + fixed position)

- *Problem:* The first version positioned the tooltip `absolute` inside its parent. Inside a card with `overflow: hidden` it's cut off; under a parent with `transform` or a lower `z-index` it's hidden behind siblings; near the screen edge it overflows off-screen.
- *Solution:* Render it in a portal on `document.body`, use `position: fixed` with viewport coordinates from `getBoundingClientRect()`, and compute the position in JS.

```
{createPortal(<div ref={tipRef} role="tooltip" className="tooltip" style={{ top, left }}>…</div>,
document.body)}
// CSS: .tooltip { position: fixed; z-index: 1000; }
```

Challenge 2: flipping and clamping inside the viewport

- *Problem:* A trigger in the top-right corner with `placement="top"` produces a tooltip partly above and to the right of the screen. Hard-coding "top" can't handle it, and naive clamping can make the arrow point at nothing.
- *Solution:* A pure function: check whether the preferred side fits, flip to the opposite side if that one fits, center on the trigger, clamp along the other axis, and compute the arrow offset from the trigger's center.

```
const placement = fits[preferred] || !fits[OPPOSITE[preferred]] ? preferred : OPPOSITE[preferred];
const left = fit(centerX - tip.width / 2, pad, viewport.width - tip.width - pad);
return { top, left, placement, arrow: fit(centerX - left, 12, tip.width - 12) };
```

Challenge 3: measuring without a flash

- *Problem:* You can only measure the tooltip after it's rendered. With `useEffect`, the browser first paints it at the old/default position (often 0,0) and then jumps, a visible flash. Measuring a `display: none` element returns 0×0.
- *Solution:* Remove `hidden`, then measure and set the position in `useLayoutEffect`, which runs after the DOM update but **before** paint; React re-renders synchronously, so the first paint is already correct. `width: max-content` keeps the measured width independent of the last position.

```
useLayoutEffect(() => {
  if (!open) return;
  updatePosition();                                  // measure + computePosition + setPosition
  window.addEventListener("scroll", updatePosition, true);
  window.addEventListener("resize", updatePosition);
  …
}, [open, updatePosition]);
```

Challenge 4: accessible, robust triggers

- *Problem:* The first version showed the tooltip only on `onMouseEnter` of a wrapper div: keyboard and screen-reader users never see it, nothing links the text to the button, there's no Escape, and moving the mouse toward the tooltip closes it. Its styles also had a duplicate `whiteSpace` key and an invalid `clip: "3"`.
- *Solution:* Show on focus too, link with `aria-describedby` (the tooltip is always in the DOM, just `hidden`), Escape to dismiss, and a hover grace period with the tooltip itself cancelling the hide.

```
<span onMouseEnter={() => schedule(true, delay)} onMouseLeave={() => schedule(false, 100)}
      onFocus={() => schedule(true, 0)} onBlur={() => schedule(false, 0)}>
  {cloneElement(children, { "aria-describedby": id })}
</span>
<div id={id} role="tooltip" hidden={!open}
     onMouseEnter={() => clearTimeout(timer.current)} onMouseLeave={() => schedule(false, 100)}>
```

8. Implement in the correct order

#	Step	Working result
1	Wrapper + `open` state + `onMouseEnter/Leave` + absolutely positioned box	Basic hover tooltip
2	`role="tooltip"`, `useId`, `cloneElement` → `aria-describedby`	Linked for screen readers
3	`onFocus/onBlur`; Escape listener	Keyboard support
4	Single `schedule()` timer: show delay, hide grace; cleanup on unmount	No flicker
5	Tooltip `onMouseEnter` cancels the hide	Hoverable
6	Portal to `document.body` + `position: fixed`	Never clipped
7	`computePosition` (place, flip, clamp, arrow) + node self-check	Correct placement
8	`useLayoutEffect` measurement; `width: max-content`	No flash, stable size
9	Scroll (capture) + resize listeners while open	Follows the trigger
10	Arrow via `--arrow` per placement class	Points at the trigger
11	Animation + `prefers-reduced-motion`; max width + wrapping	Polished
12	Demo cases: 4 sides, icon button, long text, clipped box, edge	Everything visible

9. Cover all states and edge cases

State / edge case	Handling
Closed	`hidden`; no listeners; still in the DOM for `aria-describedby`
Hover < delay	Never shows
Open via hover / via focus	Positioned before the first paint
Pointer moves onto the tooltip	Stays open
Escape	Closes; focus stays on the trigger
Hover + focus at the same time, then mouse leaves	Closes after the grace period (simple rule; see Q11)
Preferred side doesn't fit	Flips to the opposite side
Neither side fits	Keeps the preferred side, clamped (never negative)
Near a side edge	Clamped with 8px padding; arrow recalculated
Inside `overflow: hidden` / `transform` parents	Portal → unaffected
Scroll / resize while open	Repositioned
Long content	Wraps at 240px
Unmount while a timer is pending	Timer cleared
Reduced motion	No fade-in

State / edge case	Handling
Touch devices	Tapping a button focuses it (and emulates mouseenter) → shows; tapping elsewhere blurs → hides

10. Final structure

```
_15_tooltil_hover/
├── index.html
└── src/
    ├── main.jsx
    ├── index.css               ← global reset
    ├── App.jsx                 ← demo cases
    ├── App.css                 ← tooltip box, arrows, hidden, animation
    ├── utils/
    │   └── position.js         ← computePosition (pure)
    └── components/
        └── Tooltip.jsx         ← triggers, timers, portal, measurement, ARIA
```

11. Full working code

`src/utils/position.js`

```
const OPPOSITE = { top: "bottom", bottom: "top", left: "right", right: "left" };

// Keep v inside [min, max], but never below min (when the box is bigger than the space).
const fit = (v, min, max) => Math.max(min, Math.min(v, max));

// Pure positioning: where to put a tooltip of size {width, height} next to a trigger rect,
// flipping to the opposite side if the preferred side doesn't fit, and clamping inside the viewport.
export function computePosition(trigger, tip, preferred, viewport, gap = 8, pad = 8) {
  const fits = {
    top: trigger.top - tip.height - gap >= pad,
    bottom: trigger.bottom + tip.height + gap <= viewport.height - pad,
    left: trigger.left - tip.width - gap >= pad,
    right: trigger.right + tip.width + gap <= viewport.width - pad,
  };
  const placement = fits[preferred] || !fits[OPPOSITE[preferred]] ? preferred : OPPOSITE[preferred];

  if (placement === "top" || placement === "bottom") {
    const centerX = trigger.left + trigger.width / 2;
    const left = fit(centerX - tip.width / 2, pad, viewport.width - tip.width - pad);
    const top = placement === "top" ? trigger.top - tip.height - gap : trigger.bottom + gap;
    return { top, left, placement, arrow: fit(centerX - left, 12, tip.width - 12) };
  }
  const centerY = trigger.top + trigger.height / 2;
```

```
  const top = fit(centerY - tip.height / 2, pad, viewport.height - tip.height - pad);
  const left = placement === "left" ? trigger.left - tip.width - gap : trigger.right + gap;
  return { top, left, placement, arrow: fit(centerY - top, 12, tip.height - 12) };
}
```

src/components/Tooltip.jsx

```
import { cloneElement, useCallback, useEffect, useId, useLayoutEffect, useRef, useState } from "react";
import { createPortal } from "react-dom";
import { computePosition } from "../utils/position";

// <Tooltip content="Save changes" placement="top"><button>Save</button></Tooltip>
// Shows on hover (after `delay`) and on keyboard focus; hides on leave/blur/Escape.
const Tooltip = ({ content, placement = "top", delay = 300, children }) => {
  const id = useId();
  const triggerRef = useRef(null);
  const tipRef = useRef(null);
  const timer = useRef(null);
  const [open, setOpen] = useState(false);
  const [position, setPosition] = useState({ top: 0, left: 0, placement, arrow: 0 });

  const schedule = (nextOpen, wait) => {
    clearTimeout(timer.current);
    timer.current = setTimeout(() => setOpen(nextOpen), wait);
  };
  useEffect(() => () => clearTimeout(timer.current), []); // no timer firing after unmount

  const updatePosition = useCallback(() => {
    if (!triggerRef.current || !tipRef.current) return;
    const tip = tipRef.current.getBoundingClientRect();
    setPosition(
      computePosition(triggerRef.current.getBoundingClientRect(), tip, placement, {
        width: window.innerWidth,
        height: window.innerHeight,
      })
    );
  }, [placement]);

  // Layout effect: measure and place BEFORE the browser paints, so the tooltip never flashes at 0,0.
  useLayoutEffect(() => {
    if (!open) return;
    updatePosition();
    const onKeyDown = (e) => e.key === "Escape" && setOpen(false); // dismiss without moving focus
    window.addEventListener("scroll", updatePosition, true); // capture: also nested scroll containers
    window.addEventListener("resize", updatePosition);
    document.addEventListener("keydown", onKeyDown);
    return () => {
```

```
      window.removeEventListener("scroll", updatePosition, true);
      window.removeEventListener("resize", updatePosition);
      document.removeEventListener("keydown", onKeyDown);
    };
  }, [open, updatePosition]);

  return (
    <>
      <span
        ref={triggerRef}
        className="tooltip-trigger"
        onMouseEnter={() => schedule(true, delay)}
        onMouseLeave={() => schedule(false, 100)} // short grace period to move onto the tooltip
        onFocus={() => schedule(true, 0)}
        onBlur={() => schedule(false, 0)}
      >
        {cloneElement(children, { "aria-describedby": id })}
      </span>
      {createPortal(
        <div
          ref={tipRef}
          id={id}
          role="tooltip"
          hidden={!open} // always in the DOM so aria-describedby resolves, even before the first open
          className={`tooltip tooltip-${position.placement}`}
          style={{ top: position.top, left: position.left, "--arrow": `${position.arrow}px` }}
          onMouseEnter={() => clearTimeout(timer.current)} // hoverable: pointer can move onto it
          onMouseLeave={() => schedule(false, 100)}
        >
          {content}
        </div>,
        document.body
      )}
    </>
  );
};

export default Tooltip;
```

src/App.jsx

```
import "./App.css";
import Tooltip from "./components/Tooltip";

function App() {
  return (
    <main className="container">
```

```
      <h1>Tooltip</h1>
      <p className="hint">Hover with the mouse, or Tab to a button with the keyboard. Escape hides the
tooltip.</p>

      <section className="grid">
        <Tooltip content="Shown above" placement="top">
          <button type="button">Top</button>
        </Tooltip>
        <Tooltip content="Shown below" placement="bottom">
          <button type="button">Bottom</button>
        </Tooltip>
        <Tooltip content="Shown on the left" placement="left">
          <button type="button">Left</button>
        </Tooltip>
        <Tooltip content="Shown on the right" placement="right">
          <button type="button">Right</button>
        </Tooltip>
      </section>

      <section className="row">
        <span>Icon button:</span>
        <Tooltip content="Your data is encrypted at rest and in transit.">
          <button type="button" className="icon" aria-label="Security info">
            ⓘ
          </button>
        </Tooltip>
      </section>

      <section className="row">
        <span>Long text wraps at a max width:</span>
        <Tooltip content="Tooltips should stay short. This one is deliberately long to show that it
wraps instead of running off the screen.">
          <button type="button">Long tooltip</button>
        </Tooltip>
      </section>

      <section className="clip-box">
        <p>This box has overflow: hidden. The tooltip still shows fully (portal + fixed position).</p>
        <Tooltip content="Not clipped by the box" placement="top">
          <button type="button">Inside clipped box</button>
        </Tooltip>
      </section>

      <div className="edge">
        <Tooltip content="I flip and stay inside the screen" placement="right">
          <button type="button">Near the edge</button>
        </Tooltip>
```

```
      </div>
    </main>
  );
}

export default App;
```

src/App.css

```
body { background: #f9fafb; color: #111827; font-family: system-ui, sans-serif; }
.container { max-width: 640px; margin: 40px auto; padding: 0 16px 80px; }
h1 { text-align: center; font-size: 30px; }
.hint { text-align: center; color: #6b7280; margin: 6px 0 32px; }

.grid { display: flex; justify-content: center; gap: 16px; flex-wrap: wrap; padding: 40px 0; }
.row { display: flex; align-items: center; gap: 12px; margin: 16px 0; }
.clip-box { overflow: hidden; height: 110px; padding: 12px; border: 1px dashed #9ca3af; border-radius:
8px; margin: 24px 0; }
.clip-box p { font-size: 14px; color: #4b5563; margin-bottom: 10px; }
.edge { display: flex; justify-content: flex-end; }

button { padding: 8px 16px; border: none; border-radius: 8px; background: #4f46e5; color: white; font-
size: 15px; cursor: pointer; }
button:focus-visible { outline: 3px solid #a5b4fc; outline-offset: 2px; }
button.icon { width: 36px; height: 36px; padding: 0; border-radius: 50%; font-size: 18px; }

.tooltip-trigger { display: inline-block; }

.tooltip {
  position: fixed;              /* viewport coordinates from getBoundingClientRect */
  z-index: 1000;
  width: max-content;           /* size from content, not from where it was last placed */
  max-width: 240px;
  padding: 8px 12px;
  border-radius: 6px;
  background: #111827;
  color: white;
  font-size: 14px;
  line-height: 1.4;
  overflow-wrap: break-word;
  box-shadow: 0 4px 14px rgba(0, 0, 0, 0.2);
  animation: tooltip-in 120ms ease-out;
}
.tooltip[hidden] { display: none; }

/* Arrow: a rotated square at --arrow along the side facing the trigger. */
.tooltip::after { content: ""; position: absolute; width: 10px; height: 10px; background: inherit;
```

```
transform: rotate(45deg); }
.tooltip-top::after { bottom: -5px; left: calc(var(--arrow) - 5px); }
.tooltip-bottom::after { top: -5px; left: calc(var(--arrow) - 5px); }
.tooltip-left::after { right: -5px; top: calc(var(--arrow) - 5px); }
.tooltip-right::after { left: -5px; top: calc(var(--arrow) - 5px); }

@keyframes tooltip-in { from { opacity: 0; } }
@media (prefers-reduced-motion: reduce) { .tooltip { animation: none; } }
```

`src/index.css`

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: hover delay and grace period, focus/blur/Escape, hoverable tooltip, portal (no clipping), flip + clamp + arrow, no flash on open, reposition on scroll/resize, long text, timer cleanup, reduced motion, screen-reader linkage that works before the first open.

Trade-offs to mention to the interviewer

- *JS positioning vs CSS-only:* CSS-only (`position: absolute` + `:hover` / `:focus-within`) is tiny and great for simple cases, but can't flip, clamp or escape clipping. CSS Anchor Positioning (`anchor-name` , `position-try`) will solve this natively; for now JS (or Floating UI) is the reliable option.
- *Always rendering hidden tooltips:* each trigger has a hidden node in `<body>` (cheap), so `aria-describedby` always resolves. Rendering only when open saves DOM nodes but can make the first description announcement unreliable.
- *Wrapper `<span>` vs ref forwarding:* the span gets the hover/focus events (focus bubbles in React) without needing a ref on the child; the cost is one extra element (`display: inline-block`).

- `cloneElement`: simple for one child, but it overrides an existing `aria-describedby`; merge ids if the child already has one.
- *Hover delay:* avoids flicker when the mouse passes by, but delays help for intentional hovers; 300-500 ms is common, focus shows immediately.

What to add with more time

- Merge an existing `aria-describedby` on the child; support a `disabled` tooltip.
- A shared "warm-up" delay: once one tooltip opened, neighbouring ones open instantly (like toolbars).
- Click/tap mode and a `Popover` variant for interactive content (focus management, `aria-expanded`).
- Collision detection against scroll containers, not only the viewport; `ResizeObserver` for content changes.
- Tests: `computePosition` (done as a self-check) + RTL with fake timers for delay/grace/Escape and `aria-describedby`.

13. Counter questions the interviewer may ask

Requirements and design

Q1. Tooltip vs popover vs toggletip: what's the difference?

A tooltip is a short, non-interactive description shown on hover/focus (`role="tooltip"`, linked with `aria-describedby`). A popover holds interactive content (links, buttons, forms) and opens on click, with focus management. A toggletip is a tooltip opened by clicking an info button, useful on touch devices.

Q2. Should important information ever be only in a tooltip?

No. Tooltips are hard to discover on touch devices and easy to miss. Use them for supplementary hints (icon meanings, shortcuts). Required instructions, errors and labels must be visible on the page.

Q3. Can a tooltip be the only label for an icon button?

The button still needs its own accessible name (`aria-label="Security info"`). The tooltip can show the same text visually; if it's the label, link it with `aria-labelledby`, not `aria-describedby`. Don't leave an icon button nameless and rely on the tooltip.

State design

Q4. Why does the Tooltip own its `open` state instead of the parent?

So it's drop-in: no parent boilerplate, and each instance manages its own timers. Make it controllable (`open` / `onOpenChange` props) only when a parent truly needs to drive it (e.g. a guided tour).

Q5. Why a single timer for showing and hiding?

Fast enter/leave/enter sequences otherwise leave stale timers that open or close at the wrong moment. Clearing the one pending timer before scheduling the next makes the latest intent win.

Q6. Why keep both the `placement` prop and `position.placement`?

The prop is what the developer prefers; the state is what actually fits after flipping. The arrow and animation must follow the actual side.

Hard parts

Q7. Why render the tooltip in a portal?

Ancestors with `overflow: hidden/auto` clip their children, and ancestors with `transform`, `filter` or `z-index` create stacking contexts that can bury it. In `document.body` with `position: fixed`, nothing can clip or cover it, and React events still bubble through the React tree.

Q8. Why `useLayoutEffect` instead of `useEffect` for positioning?

`useLayoutEffect` runs after React updates the DOM but before the browser paints, so you can measure and set the final position, and the first frame is already correct. With `useEffect`, the user may see a frame at the wrong position (a "flash"/jump).

Q9. How does flipping work?

Check whether the tooltip fits on the preferred side (with gap and padding). If not, and the opposite side fits, use the opposite. If neither fits, keep the preferred side and clamp. Then center on the trigger and clamp along the other axis.

Q10. Why listen to scroll with `capture: true`?

`scroll` doesn't bubble. A capture listener on `window` receives scroll events from **any** scrollable ancestor (a scrolling sidebar or table), not only the page, so the tooltip follows the trigger everywhere.

Q11. A user focuses the trigger with the keyboard and also hovers it, then moves the mouse away. What should happen?

Ideally it stays open while either hover or focus is active. Track them separately (`hovered || focused`) instead of one boolean. Our simpler version closes on mouse leave; mention the improvement if asked.

Implementation steps

Q12. What does WCAG 1.4.13 (Content on Hover or Focus) require?

The extra content must be **dismissible** (Escape closes it without moving the pointer or focus), **hoverable** (the pointer can move onto it without it disappearing), and **persistent** (it stays until hover/focus leaves or it's dismissed, no auto-timeout). Our grace period, tooltip hover handlers and Escape cover all three.

Q13. How do you test a tooltip?

Unit-test `computePosition` (fits, flips, clamps, arrow). With RTL + fake timers: hover → advance 299 ms → not visible; 1 ms more → visible with `role="tooltip"`; the trigger's `aria-describedby` equals the tooltip id; Escape hides it; focus shows it immediately; unmount with a pending timer → no warning. Use Playwright for real positioning.

Q14. How do you handle touch devices?

Hover doesn't exist; a tap emulates mouseenter and focuses buttons, so a focus-triggered tooltip appears and tapping elsewhere hides it. For critical help on mobile, use a toggletip (tap an ⓘ button to toggle) or inline text.

Performance and scale

Q15. A table has 500 cells with tooltips. Any concerns?

500 hidden portal nodes plus 500 wrapper spans is acceptable but wasteful. Better: a single shared tooltip that reads its content from the hovered element (event delegation with `data-tooltip` attributes), rendered once; or render the tooltip node only while open and use `aria-label` / visually hidden text for accessibility.

Q16. Why not reposition on every `mousemove`?

The tooltip anchors to the trigger, not the cursor. Recompute only when the trigger can move: on open, scroll and resize (and with a `ResizeObserver` if the content changes). Cursor-following tooltips need rAF throttling.

Q17. How would you avoid re-renders for every scroll event while open?

Write the position directly to the tooltip's style in the scroll handler (refs, no `setState`), throttled with `requestAnimationFrame`, or use CSS Anchor Positioning so the browser does it.

Trade-offs and "why not X?"

Q18. Why not the `title` attribute?

`title` is free, but it shows only after a long, uncontrollable delay, can't be styled, doesn't appear on keyboard focus in most browsers or on touch, and screen readers handle it inconsistently. Fine for low-stakes hints, not for a design-system tooltip.

Q19. Why not Floating UI / Popper / Radix Tooltip?

In production, yes: Floating UI handles flip, shift, arrow, scroll containers, virtual elements and many edge cases; Radix adds the full ARIA behaviour. In an interview, writing `computePosition` shows you understand viewport math, portals and layout timing.

Q20. What is CSS Anchor Positioning, and would it replace this?

A newer CSS feature: the trigger gets `anchor-name: --t`, the tooltip `position-anchor: --t` with `position-area: top`, and `position-try-fallbacks: flip-block` flips automatically, with no JS measuring. Combined with the HTML `popover` attribute (top layer), it will replace most JS positioning once browser support is broad enough.

Q21. Why `cloneElement` instead of asking users to pass `aria-describedby` themselves?

It makes the component impossible to misuse: every trigger is linked automatically. The drawbacks: it only works with a single element child, and it overrides an existing `aria-describedby` (merge the ids to fix that).